

Redes y Comunicaciones

Ing. Mariano Coccia

Instituto Superior Juan XXIII

Capítulo 2: Capa Aplicación

Capítulo 2: Capa Aplicación

2.1 Principios de las aplicaciones de red

2.2 Web y HTTP

2.3 FTP

2.4 Correo Electrónico
SMTP, POP3, IMAP

2.5 DNS

Capítulo 2: Capa Aplicación

Objetivos:

- Veremos los aspectos conceptuales y de implementación de los protocolos de aplicación
  - Modelo de servicio de la capa transporte
  - Paradigma cliente-servidor
  - Paradigma peer-to-peer (par-a-par o entre pares)
- Aprendizaje de protocolos examinando protocolos de aplicación populares
  - HTTP
  - FTP
  - SMTP / POP3 / IMAP
  - DNS

Algunas aplicaciones de red

- E-mail
- Web
- Mensajería instantánea
- Login remoto
- Compartir archivos P2P
- Juegos de red multiusuarios
- Reproducción de clips de video almacenados
- Telefonía Internet (VoIP)
- Conferencias de video en tiempo real
- Computación paralela masiva.

Creación de una aplicación de red

Aplicaciones de la red

- Corren en diferentes sistemas y se comunican por la red.
- Ej. Web: Programa del servidor Web se comunica con el programa del navegador

No se refiere a software escrito para los dispositivos de red

- Dispositivos de la red (routers, switches) no funcionan en la capa aplicación

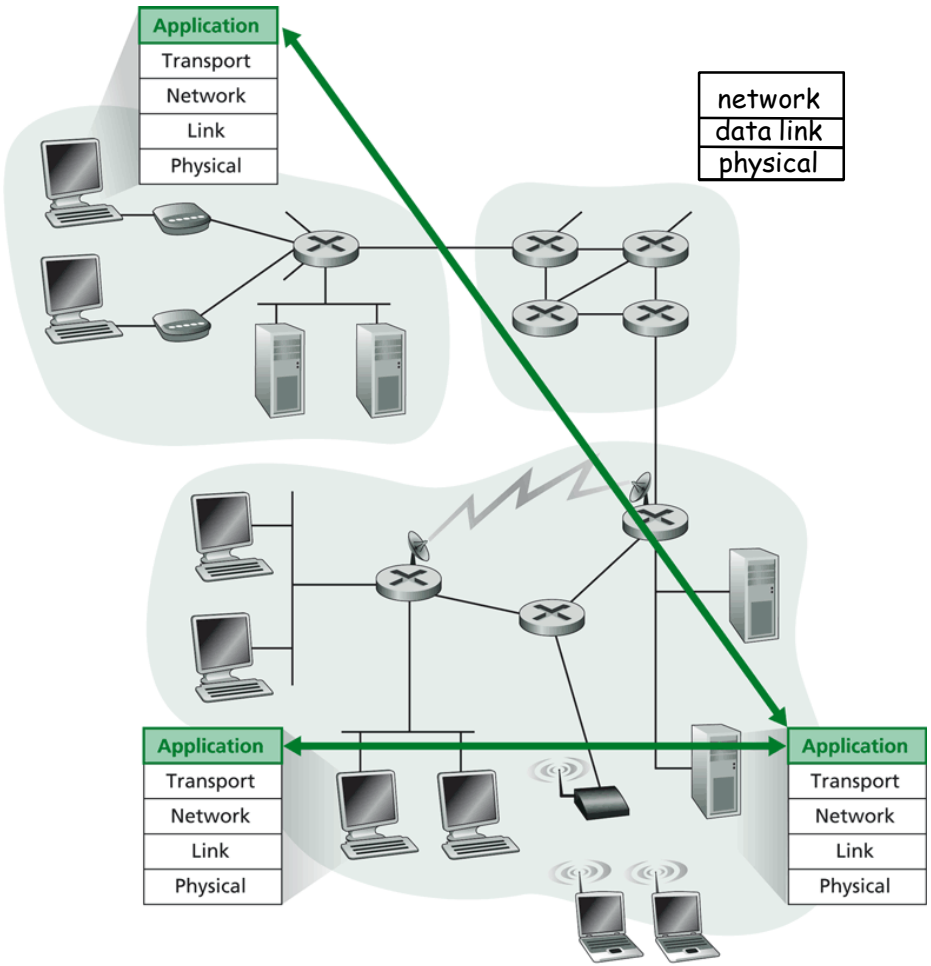


Figure 2.1 ♦ Communication for a network application takes place between end systems at the application layer.

Capítulo 2: Capa Aplicación

2.1 Principios de las aplicaciones de red

2.2 Web y HTTP

2.3 FTP

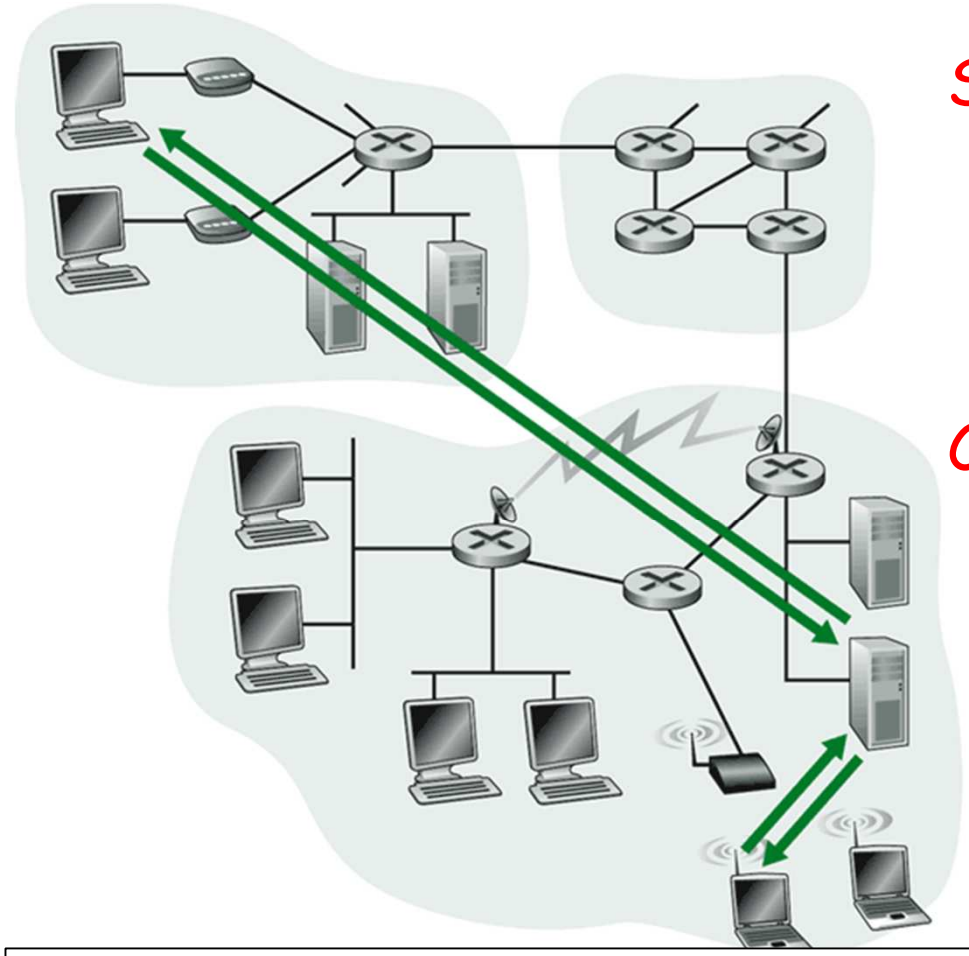
2.4 Correo Electrónico
SMTP, POP3, IMAP

2.5 DNS

Arquitecturas de Aplicación

- Cliente-servidor
- Peer-to-peer (P2P)
- Híbridos de cliente-servidor y P2P

Arquitectura Cliente-servidor



Servidor:

- Computador siempre encendido
- Dirección IP permanente
- Granja de servidores por **escalamiento**

Cliente:

- Se comunica con servidor
- Puede conectarse intermitentemente
- Puede tener direcciones IP dinámicas (variable)
- No se comunican directamente entre sí (dos clientes puros)

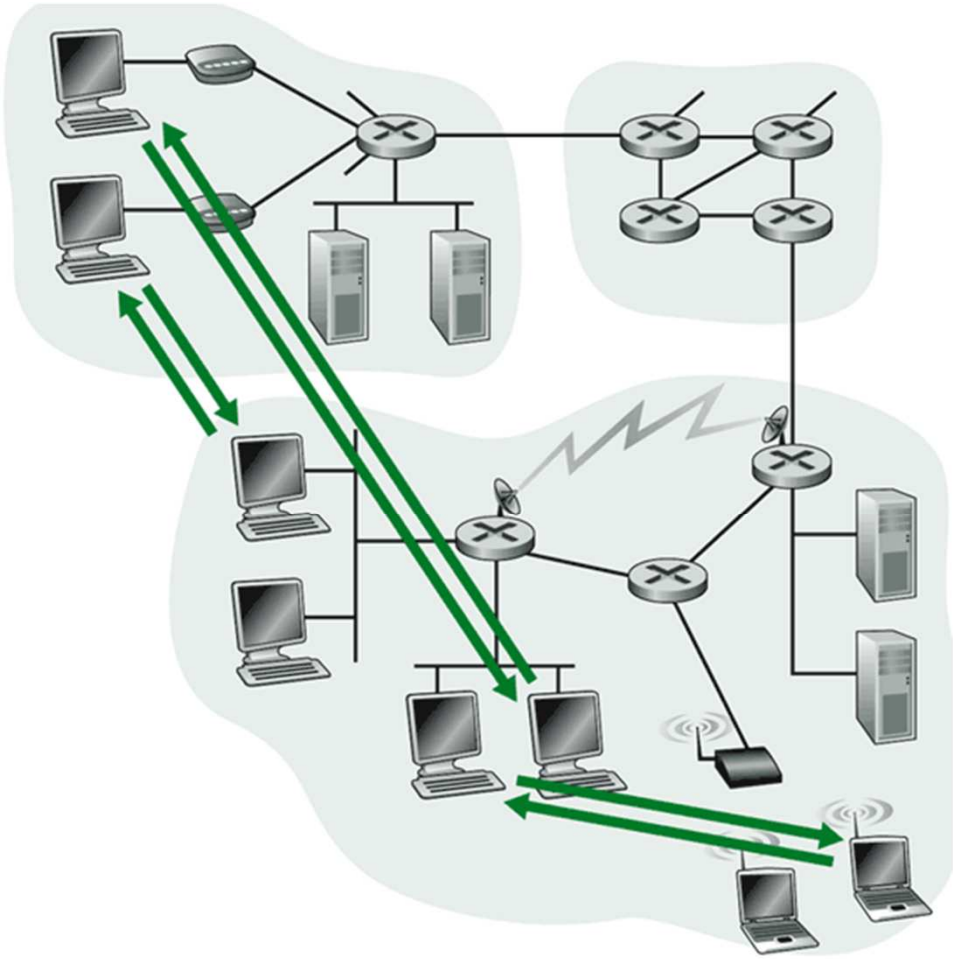
Escalabilidad: es la habilidad de extender la operación (más clientes) sin perder calidad. Ej. Radio

Arquitectura P2P Pura

- No hay servidor
- Sistemas terminales arbitrarios se comunican directamente
- Pares se conectan intermitentemente y cambian sus direcciones IP
- Ejemplo: Gnutella, Torrent

Altamente escalable

Pero difícil de administrar



b. Peer-to-peer application

Híbridos de cliente-servidor y P2P

Napster

- Transferencia de archivos P2P
- Búsqueda de archivos centralizada:
  - Clientes registran contenidos en servidor central
  - Clientes consultan algún servidor central para localizar el contenido

Mensajería Instantánea

- Diálogo es entre los usuarios es P2P (no pasa por servidor)
- Detección/localización de presencia es centralizada:
  - Usuario registra su dirección IP en un servidor central cuando ingresa al sistema
  - Usuarios contactan servidor central para encontrar las direcciones IP de sus amigos.

Procesos que se comunican

Proceso: es un programa corriendo en un computador.

- Dentro de la máquina dos procesos se comunican usando comunicación entre procesos (definida por Sistema Operativo).
- Procesos en diferentes hosts se comunican vía intercambio de mensajes

Definiciones

Proceso Cliente: proceso que inicia la comunicación

Proceso servidor: proceso que espera ser contactado

- Nota: En aplicaciones con arquitectura P2P un mismo proceso actúa como cliente y servidor

Sockets

- Un proceso envía/recibe mensajes a/desde su socket
- socket es un punto de comunicación entre dos partes (análogo a una puerta)
  - Proceso transmisor envía mensajes por un socket
  - Proceso transmisor confía en la infraestructura de transporte al otro lado de la puerta, la cual lleva los mensajes al socket en el proceso receptor

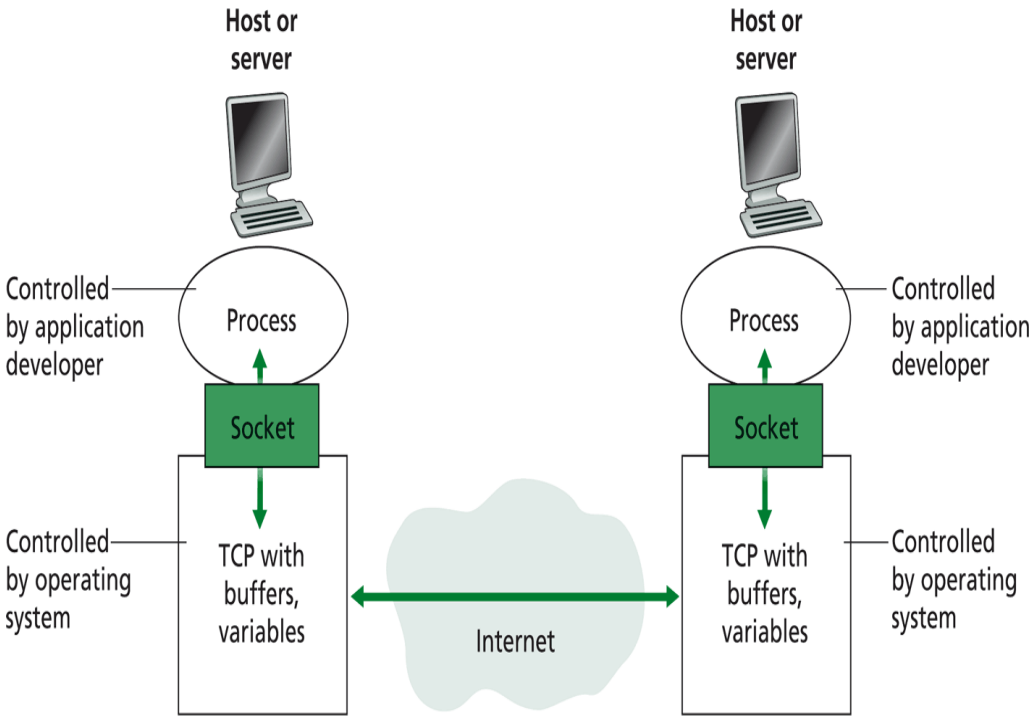


Figure 2.3 ♦ Application processes, sockets, and underlying transport protocol

- API: Interfaz de Programación de Aplicaciones
  Los lenguajes ofrecen mecanismos para comunicarse con el sistema operativo y la capa de transporte.
  (volveremos más adelante)

Direccionamiento de procesos

- Para que un proceso reciba un mensaje, éste debe tener una forma para identificarlo (diferenciarlo)
- Un terminal/host tiene al menos una dirección IP única de 32 bits.
- Q: ¿Es suficiente la dirección IP para identificar un proceso en un host?
- Respuesta: No, muchos procesos pueden estar corriendo en el mismo host (= computador).

- El identificador de proceso incluye la dirección IP y un número de puerto (port) asociado con el proceso en el host.
- Ejemplo de número de puerto (port number):
  - Servidor HTTP (web): 80
  - Servidor SMTP (mail): 25

Viendo las conexiones de nuestra máquina

- En Linux y Windows netstat (Network Statistic):
  - **netstat –p tcp para ver conexiones TCP**
  - **netstat –p udp para ver conexiones UDP**

- Hasta aquí sabemos:
  - Relación entre aplicación y proceso
  - Necesidad de los puertos
  - Mecanismo de software usado para pedir servicios a capa transporte
- Ahora estudiaremos algunos protocolos.

Protocolos de capa aplicación definen:

- Tipos de mensajes intercambiados:
  mensajes de requerimiento y respuesta
- Sintaxis de los mensajes:
  los campos en los mensajes y cómo éstos son delimitados.
- Semántica de los campos:
  significado de la información en los campos
- Reglas para cuándo y cómo los procesos envían y responden a mensajes

Protocolos de dominio público:

- Definidos en RFCs
- Permite interoperatividad
- Ej: HTTP (WEB), SMTP (email), POP3 (email)

Protocolos propietarios:

- KaZaA
- Skype

¿Qué servicios de la capa transporte necesita una aplicación?

Confiabilidad en la entrega (Sin pérdida de datos)

- Algunas aplicaciones (ej., transferencia de archivos, telnet) requieren transferencia 100% confiable
- otras (ej., audio) pueden tolerar pérdida

Retardo

- algunas Aplicaciones (ej., Telefonía en internet, juegos interactivos) requieren bajo retardo para ser "efectivas"

Tasa de datos ("Bandwidth")

- algunas aplicaciones (ej., multimedia) requieren cantidad mínima de ancho de banda para ser "efectivas"
- otras ("aplicaciones elásticas") hacen uso del bandwidth que obtengan

Requerimientos de servicios de transporte de aplicaciones comunes

Aplicación	Pérdidas	Bandwidth	Sensible a Tiempo
file transfer	no	elastic	no
e-mail	no	elastic	no
Web documents	no	elastic	no
real-time audio/video	tolerante	audio: 5kbps-1Mbps video:10kbps-5Mbps	yes, 100's msec
stored audio/video	tolerante	Igual al de arriba	yes, few secs
interactive games	tolerante	few kbps up	yes, 100's msec
instant messaging	no	elastic	yes and no

Servicios de los protocolos de transporte en Internet

Servicio TCP

(Transmission Control Protocol):

- *Es Orientado a la conexión* establecer conexión (setup) requerido entre procesos cliente y servidor antes de transferencia
- *Ofrece Transporte confiable* entre proceso Transmisor (Tx) y Receptor (Rx)
- *Tiene Control de flujo:* Tx no sobrecargará al Rx
- *Tiene Control de congestión:* el Tx se frena cuando la red está sobrecargada
- *No provee:* garantías de retardo ni ancho de banda mínimos

Servicio UDP

(User Datagram Protocol):

- Transferencia de datos no confiable entre proceso Tx y Rx.
- No provee: establecimiento conexión, confiabilidad, control de flujo, control de congestión, garantías de retardo o ancho de banda

¿Por qué existe UDP?

Aplicaciones Internet: aplicación, protocolo de transporte

Aplicación	Protocolo capa aplicación	Protocolo de transporte que lo sustenta
e-mail	SMTP [RFC 2821]	TCP
remote terminal access	Telnet [RFC 854]	TCP
Web	HTTP [RFC 2616]	TCP
file transfer	FTP [RFC 959]	TCP
streaming multimedia	proprietary (e.g. RealNetworks)	TCP or UDP
Internet telephony	proprietary (e.g., Dialpad, skype)	typically UDP

Capítulo 2: Capa Aplicación

2.1 Principios de las aplicaciones de red

2.2 Web y HTTP

2.3 FTP

2.4 Correo Electrónico
SMTP, POP3, IMAP

2.5 DNS

Web y HTTP

- Una página Web está compuesta de objetos
- En este contexto objetos pueden ser archivos HTML, imágenes (JPEG, GIF,...), Java applets, archivos de audio, archivos de vídeo,...
- Páginas Web consisten generalmente de un archivo HTML base el cual incluye referencias a objetos.
- Cada objeto es direccionable por un Universal Resource Locator (URL)
- Ejemplo URL:

```
http://www.elo.utfsm.cl:80/imgmenu/header.jpg
```

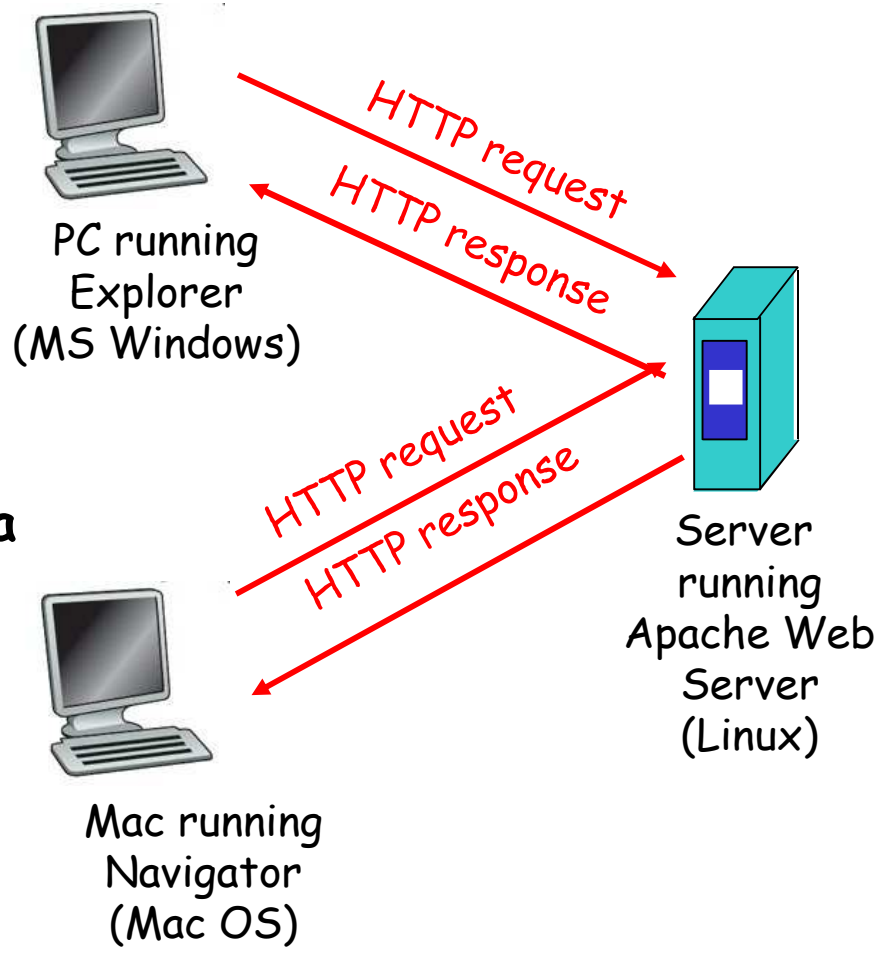
Nombre de la máquina y puerto

Nombre de camino (path name)

HTTP Generalidades

HTTP: hypertext transfer protocol

- Protocolo de la capa aplicación usado por la Web
- Modelo cliente/servidor
  - *cliente:* browser primero requiere y luego recibe y "despliega" objetos Web
  - *servidor:* Servidor Web envía objetos en respuesta a requerimientos
- HTTP 1.0: RFC 1945 (1996)
- HTTP 1.1: RFC 2068 (1997)
- HTTP 1.1 Mejorado RFC 2616 (1999)



HTTP generalidades (cont.)

Usa TCP:

1) Cliente inicia conexión TCP (crea socket) al servidor, puerto 80 (puede ser otro!)
2) Servidor acepta conexión TCP del cliente
3) Mensajes HTTP (mensajes del protocolo de capa aplicación) son intercambiados entre browser (cliente HTTP) y servidor Web (servidor HTTP)
4) Se cierra la conexión TCP

HTTP no guarda "estado"

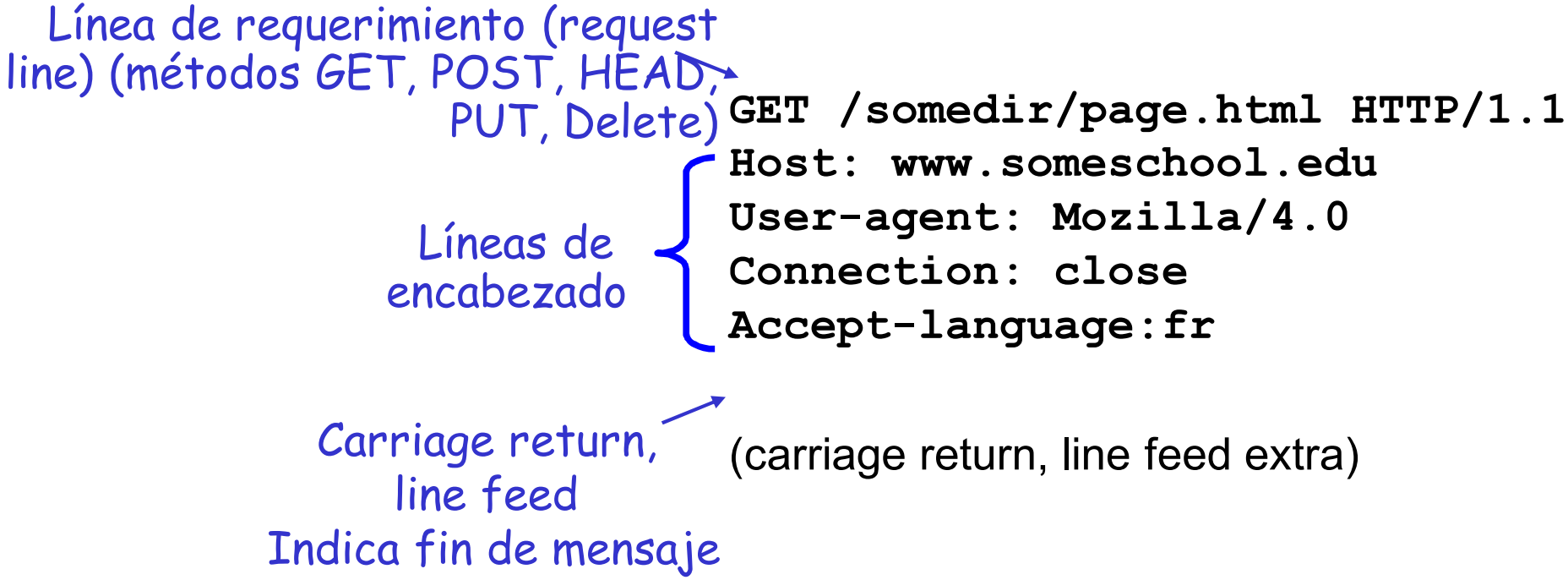
- El servidor no mantiene información sobre los requerimientos del clientes

Protocolos que mantienen "estado" son complejos!

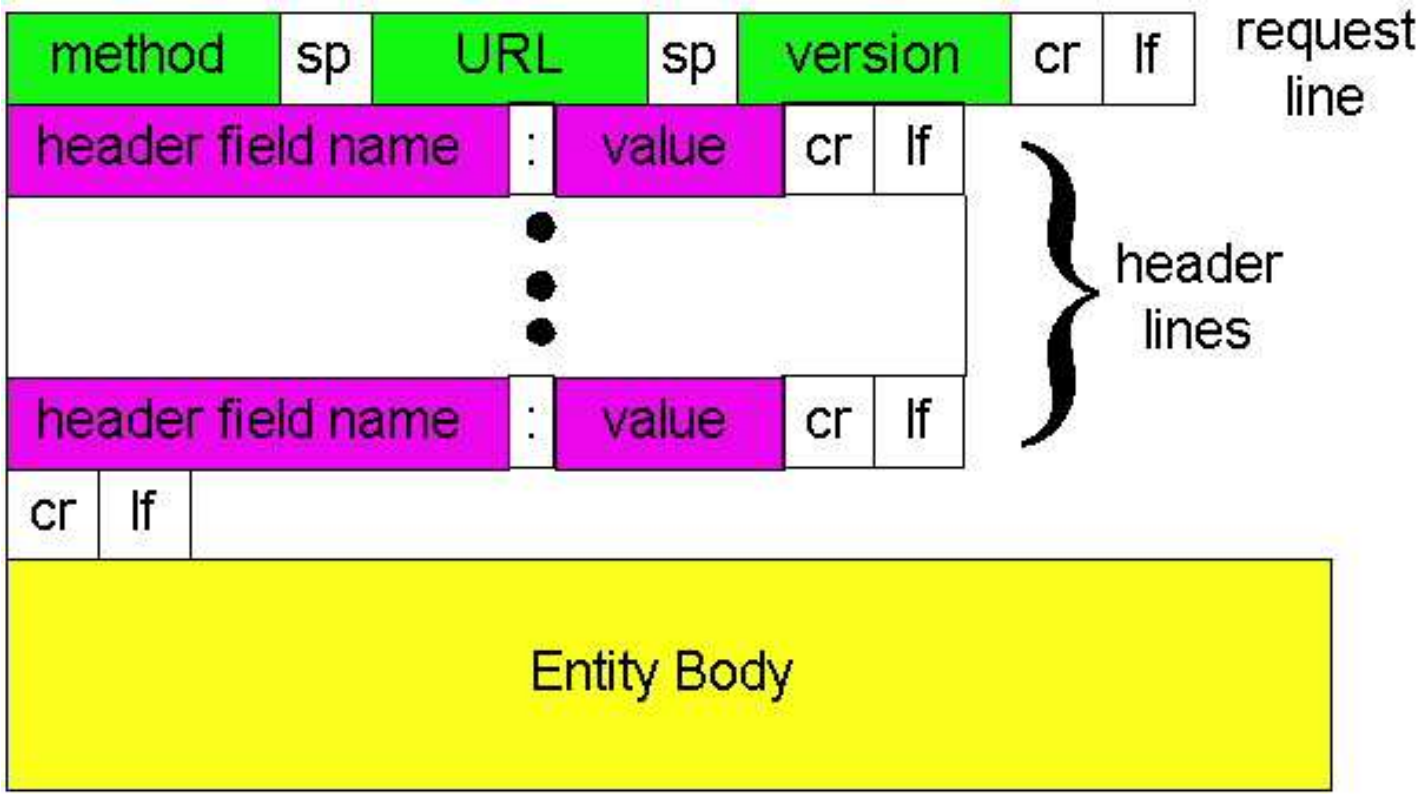
- Historia pasada (estado) debe ser mantenida
- Si servidor o cliente se cae, las vistas del estado pueden ser inconsistentes, y deben ser sincronizadas

Mensaje HTTP de requerimiento

- Dos tipos de mensajes HTTP: *requerimiento y respuesta*
- **Mensaje de requerimiento HTTP:**
  - **ASCII (es decir, formato legible)**



Mensaje HTTP de requerimiento: formato general



Tipos de Métodos

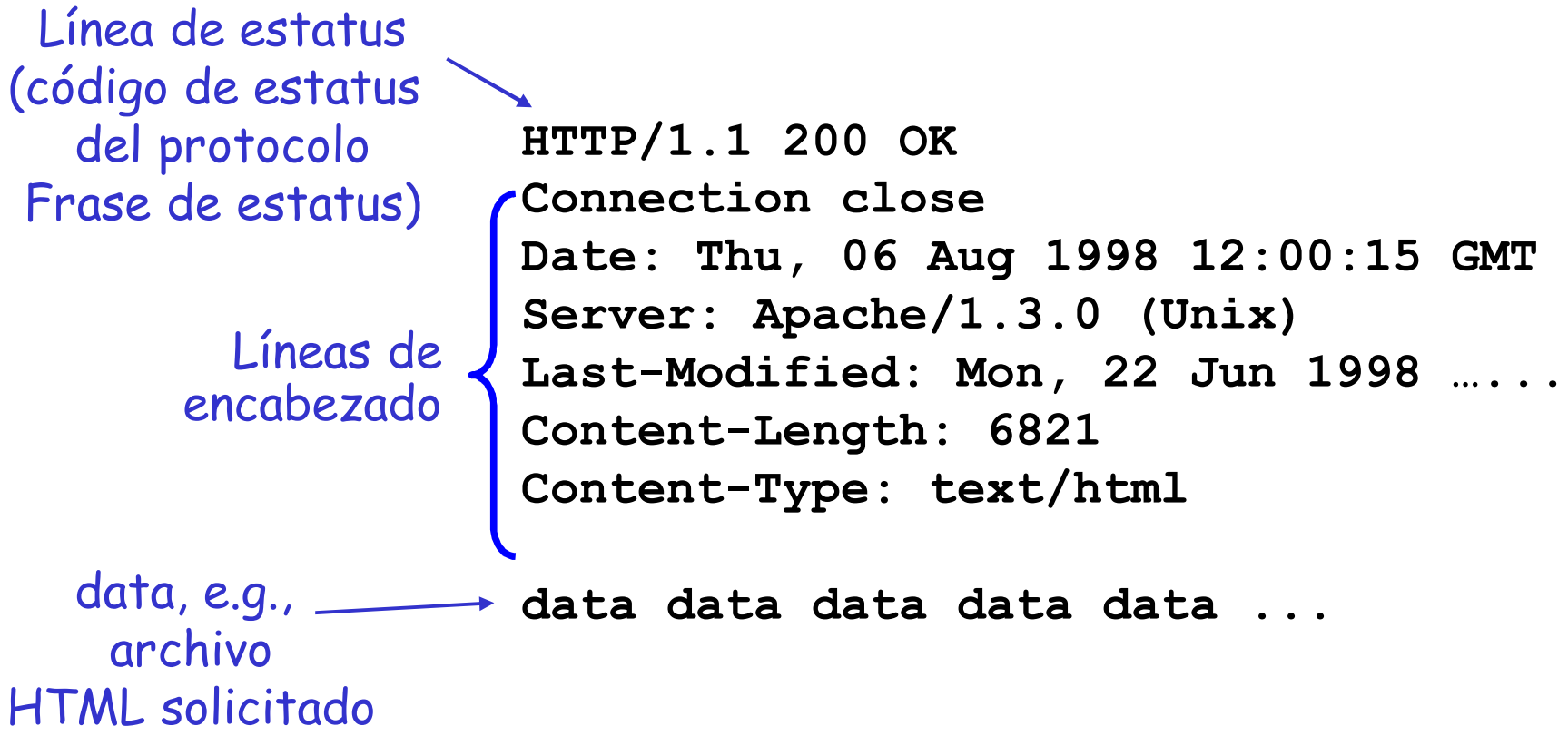
HTTP/1.0

- GET
- POST
- HEAD
  - Pide al servidor que deje el objeto requerido afuera de la respuesta. Respuesta incluye sólo el encabezado.

HTTP/1.1

- GET, POST, HEAD
- PUT
  - Sube archivos en cuerpo del requerimiento en localización indicada por el campo URL
- DELETE
  - Borra archivo especificado en el campo URL

Mensajes HTTP de respuesta



Códigos HTTP de respuesta

En primera línea de respuesta del servidor-> cliente.

Algunos códigos de muestra:

200 OK

- request exitoso, objeto requerido es incluido luego en mensaje

301 Moved Permanently

- Se movió el objeto requerido, nueva ubicación es especificada luego en el mensaje (Location:)

400 Bad Request

- Requerimiento no entendido por el servidor

404 Not Found

- Documento no encontrado en servidor

505 HTTP Version Not Supported

- Versión de HTTP no soportada

Probando HTTP (lado cliente)

1. Telnet a tu servidor favorito:

```
telnet profesores.elo.utfsm.cl 80
```

Telnet abre una conexión TCP al puerto 80 (puerto servidor HTTP por omisión) en mateo.elo.utfsm.cl. Cualquier cosa ingresada es enviada a puerto 80 de mateo

2. Escribir un requerimiento GET HTTP:

```
GET /~agv/elo322/1s09/prueba.html HTTP/1.1
Host: profesores.elo.utfsm.cl

NOTA: Campo Host es obligatorio
en encabezado, requerido por proxy.
```

Tipeando esto (doble carriage return), enviamos un GET request mínimo (pero completo) al servido HTTP

3. Observar el mensaje de respuesta enviado por el servidor HTTP!

Hacer algo similar con navegador y wireshark

Capítulo 2: Capa Aplicación

2.1 Principios de las aplicaciones de red

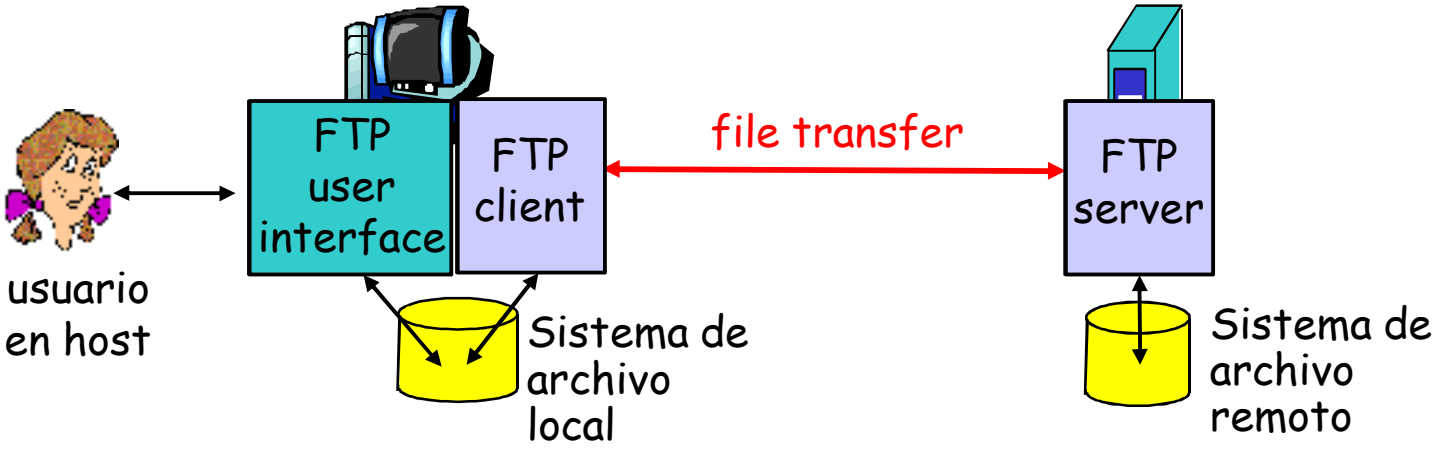
2.2 Web y HTTP

2.3 FTP

2.4 Correo Electrónico
SMTP, POP3, IMAP

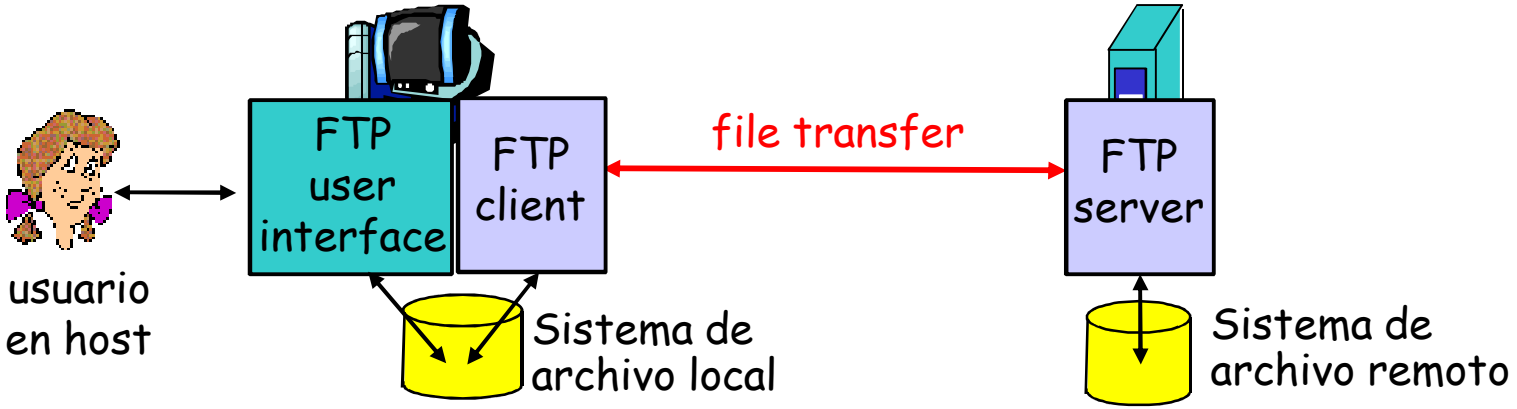
2.5 DNS

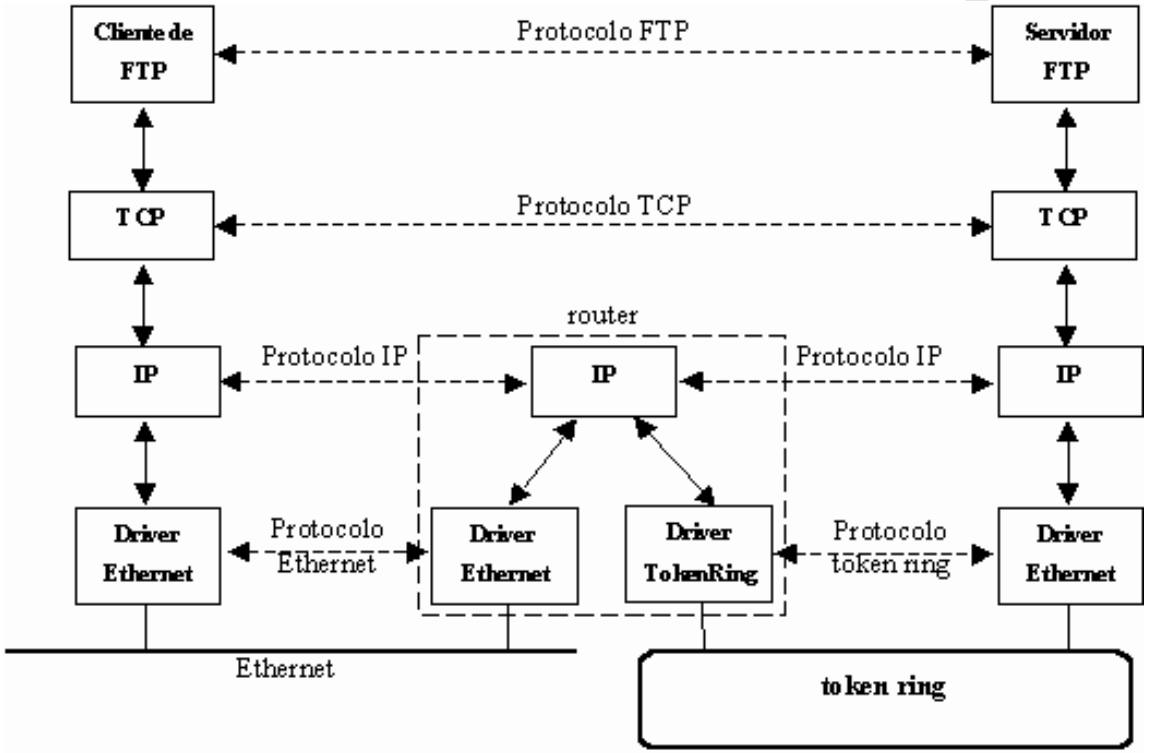
FTP: El protocolo de transferencia de archivos (File Transfer Protocol)



- Transferencia de archivos a/desde el host remoto
- Sigue modelo cliente/servidor
  - *cliente:* sitio que inicia la transferencia (ya sea a/desde sitio remoto)
  - *servidor:* host remoto
- ftp: RFC 959
- Servidor ftp: puerto 21, cliente en cualquier puerto. Ver netstat –p tcp

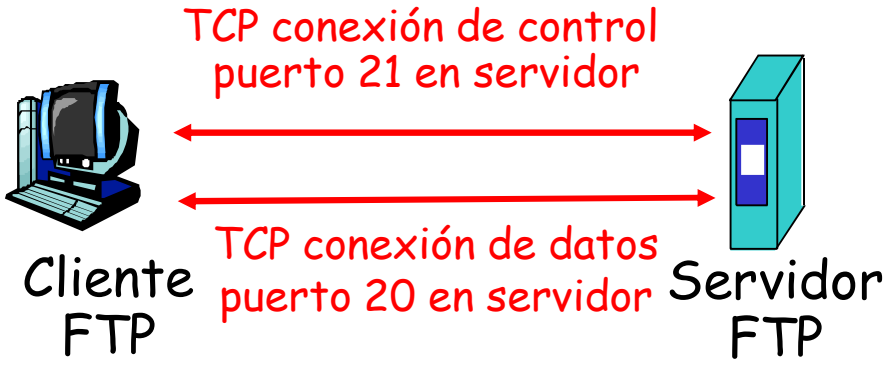
FTP: La capa aplicación se apoya en la inferiores.





FTP: Conexiones separadas de control y datos

- Cliente FTP contacta servidor FTP en puerto 21, especificando TCP como protocolo de transporte
- El cliente obtiene autorización sobre el control de la conexión
- El cliente navega el directorio remoto enviando comandos sobre la conexión de control.
- Cuando el servidor recibe una petición de transferencia de archivo, el servidor abre una conexión de datos hacia el cliente. Éste es Modo Activo.
- Después de la transferencia de un archivo, el servidor cierra la conexión de datos.



- El servidor abre una segunda conexión TCP de datos para transferir otro archivo.
- Conexión de control: "out of band" (fuera de banda)
- Servidor FTP mantiene "estado": directorio actual, cuenta de usuario conectado.
- Existe modo activo y pasivo

FTP comandos, respuestas

Muestra de comandos:

- Son enviados como texto ASCII vía el canal de control
- `USER` *username*
- `PASS` *password*
- `LIST` retorna la lista de archivos del directorio actual
- `RETR filename` baja un archivo (gets)
- `STOR filename` almacena (puts) archivo en host remoto

Algunos códigos retornados

- Código status y frases (como en HTTP)
- `331 Username OK, password required`
- `125 data connection already open; transfer starting`
- `425 Can't open data connection`
- `452 Error writing file`

Pregunta:

Dé una razón por la que hoy la aplicación "ftp" no es recomendada para transferencia de archivos. ?

- Ftp no encripta su tráfico, tanto el password como los datos transferidos pueden ser leídos al tener acceso a los paquetes de la conexión.
- A cambio de ftp usted puede usar hoy sftp, es su equivalente con transferencias encriptadas.

Capítulo 2: Capa Aplicación

2.1 Principios de las aplicaciones de red

2.2 Web y HTTP

2.3 FTP

2.4 Correo Electrónico
SMTP, POP3, IMAP

2.5 DNS

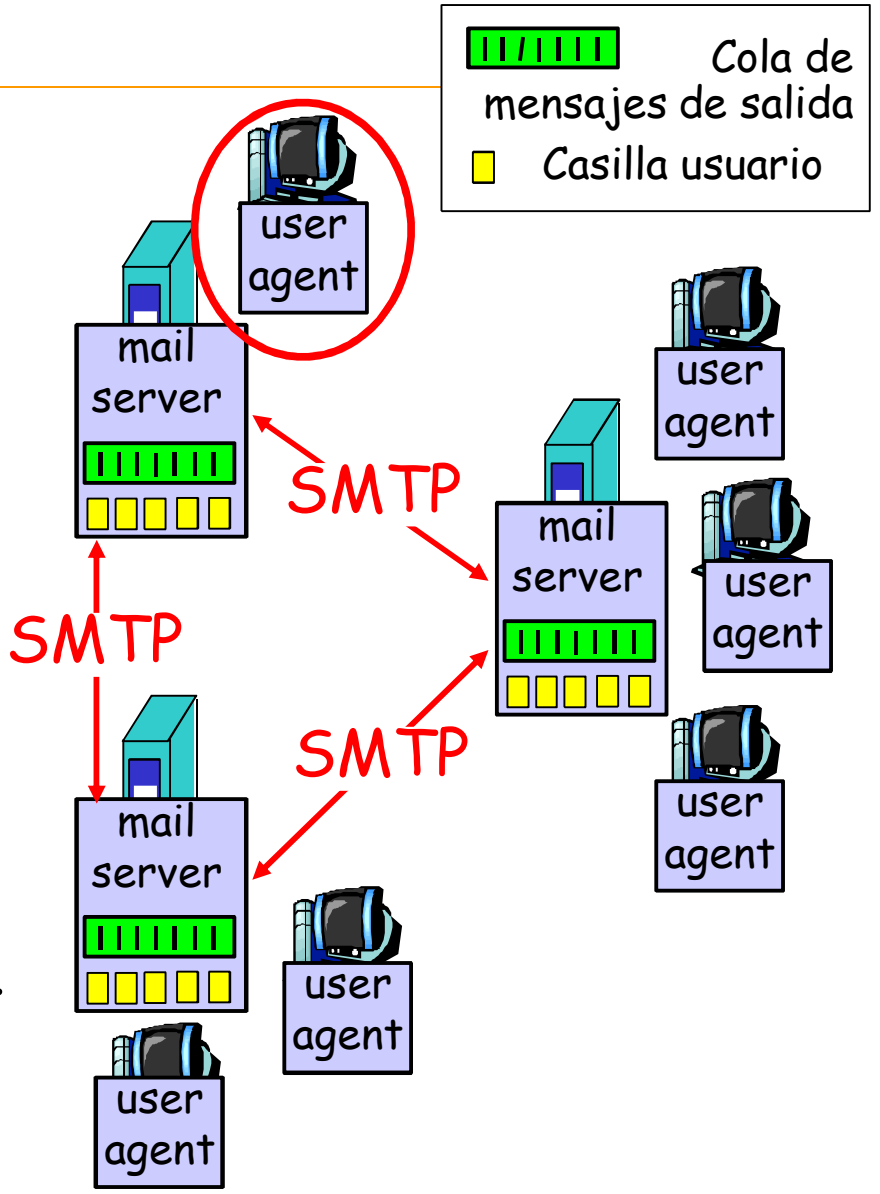
Correo Electrónico

Tres mayores componentes:

- Agente usuario o cliente de correo
- Servidor de correo
- Simple Mail Transfer Protocol: SMTP

Agente Usuario

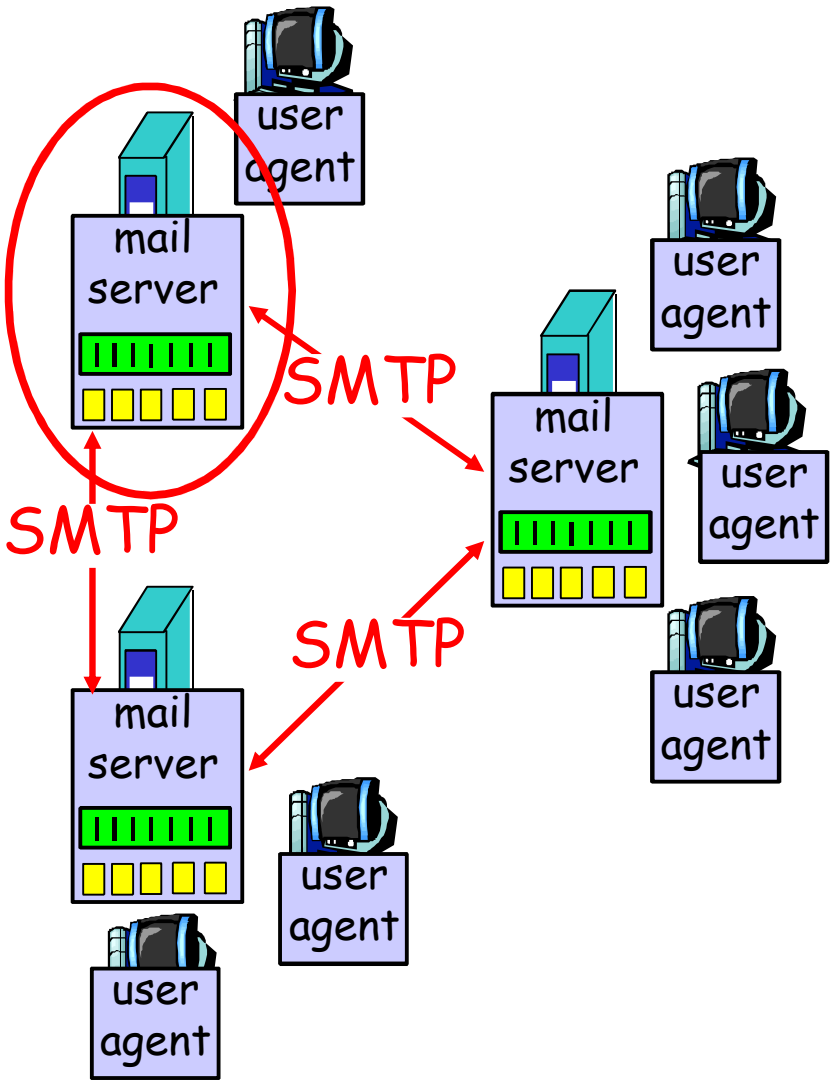
- También conocido como "cliente de correo"
- Escritura, edición, lectura de mensajes de correos
- Ej. Eudora, Outlook, Mozilla Thunderbird, Iphone mail client
- Mensajes de salida y entrada son almacenados en servidor



Correo Electrónico: Servidor de correo

Servidor de Correo

- **Casilla** contiene mensajes de entrada para el usuario
- **Cola de mensajes** de los correos de salida
- **SMTP: Protocolo** entre servidores de correo para enviar mensajes e-mail
  - "cliente": servidor que envía el correo
  - "servidor": servidor que recibe el correo
- También lo usa el agente usuario para enviar correo.



Correo Electrónico: SMTP [RFC 2821]

- Usa TCP para transferir confiablemente mensajes e-mail desde el cliente al servidor, puerto 25 en servidor.
- Transferencia directa: servidor envía correos al servidor receptor
- Tres fases de la transferencia
  - handshaking (apretón de manos para establecer conexión)
  - transferencia de mensajes
  - cierre
- Interacción comandos/respuestas
  - comandos: Texto ASCII
  - respuesta: código de status y frase.
- Mensajes deben ser enviados en ASCII de 7-bits ¿Qué pasa con las fotografías y archivos binarios?

Escenario: Alicia envía mensaje a Bob

1) Alicia usa èl cliente de mail para componer el mensaje para `bob@someschool.edu`

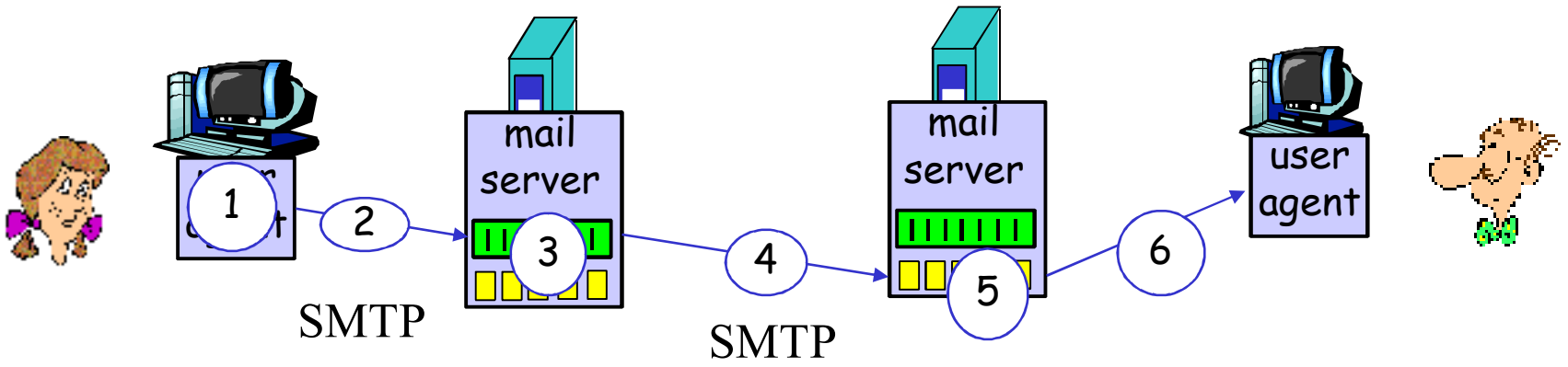
2) El cliente de Alicia envía en mensaje a su servidor de correo; el mensaje es puesto en cola de salida

3) Lado cliente de SMTP abre una conexión TCP con el servidor de correo de Bob

4) El cliente SMTP envía el mensaje de Alicia por la conexión TCP

5) El servidor de correo de Bob pone el mensaje en su casilla

6) Bob invoca su cliente de mail para leer el mensaje



Prueba de interacción SMTP (obsoleta)

- `telnet servername 25`
- Ver respuesta 220 desde el servidor
- Ingresar los comandos HELO, MAIL FROM, RCPT TO, DATA, QUIT

Lo de arriba nos permite enviar correo sin usar el cliente de correo.



- Hoy muchos servidores están configurados para aceptar sólo conexiones seguras que no permiten el uso de telnet para envío de correo. Por ejemplo, los servers de gmail usan TLS (Transport Layer Security)

Ejemplo de Interacción SMTP

Luego de: $telnet hamburger.edu 25 <enter>

S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection

En el pasado esto era posible. Hoy los servidores utilizan conexiones seguras, telnet no funciona.

Formato de mensajes de correo (comando DATA)

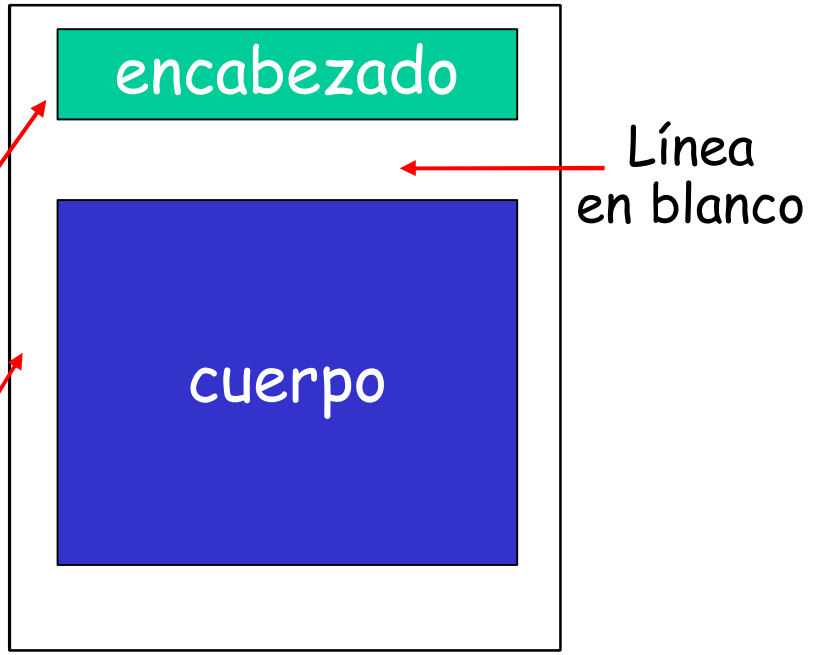
SMTP: protocolo para intercambio de mensajes de correo

RFC 822: estándar para el formato de los mensajes:

- Ej. líneas de encabezado (opcional), entre otros:
  - To:
  - From:
  - Subject:

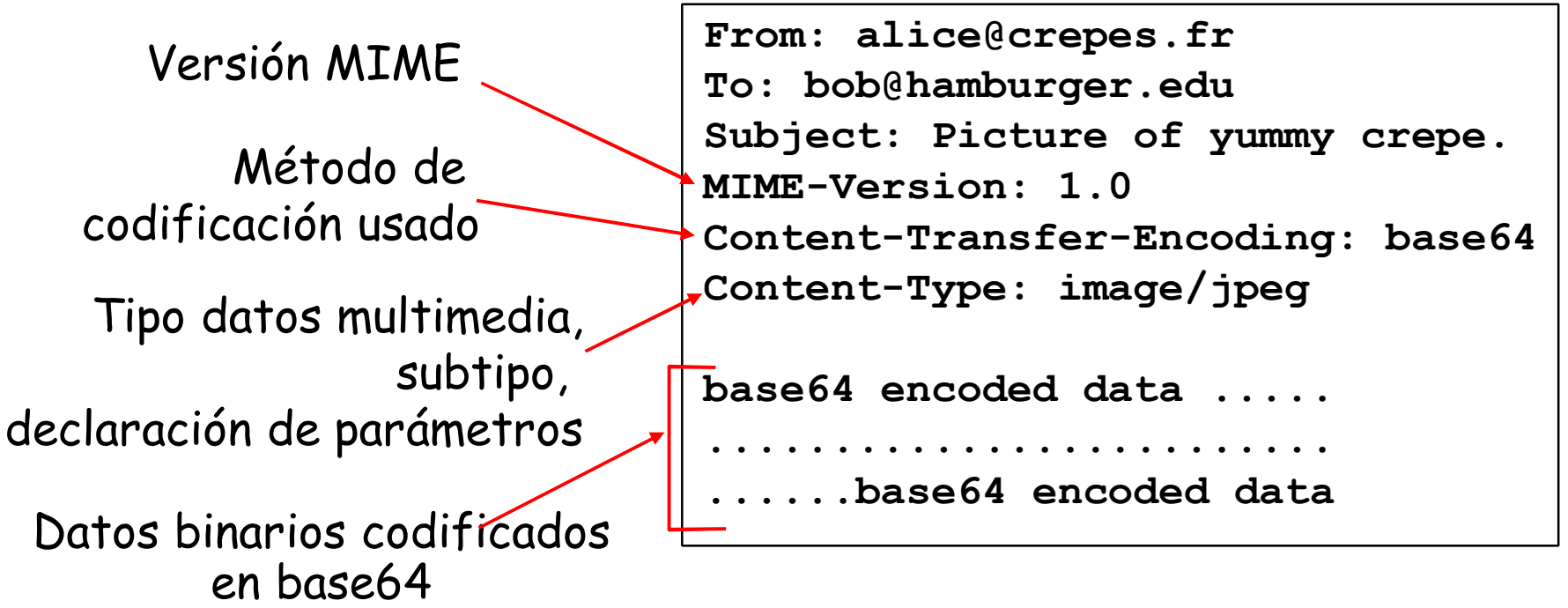
diferente a los comandos SMTP!

- Cuerpo
  - El "mensaje", sólo caracteres ASCII



Formato de mensaje: extensiones multimedia

- MIME: "multimedia mail extension", RFC 2045, 2056
- Líneas adicionales en el encabezado del mensaje declaran el tipo de contenido MIME
- La codificación Base64 usa sólo los caracteres: A-Z, a-z, 0-9 y +/=



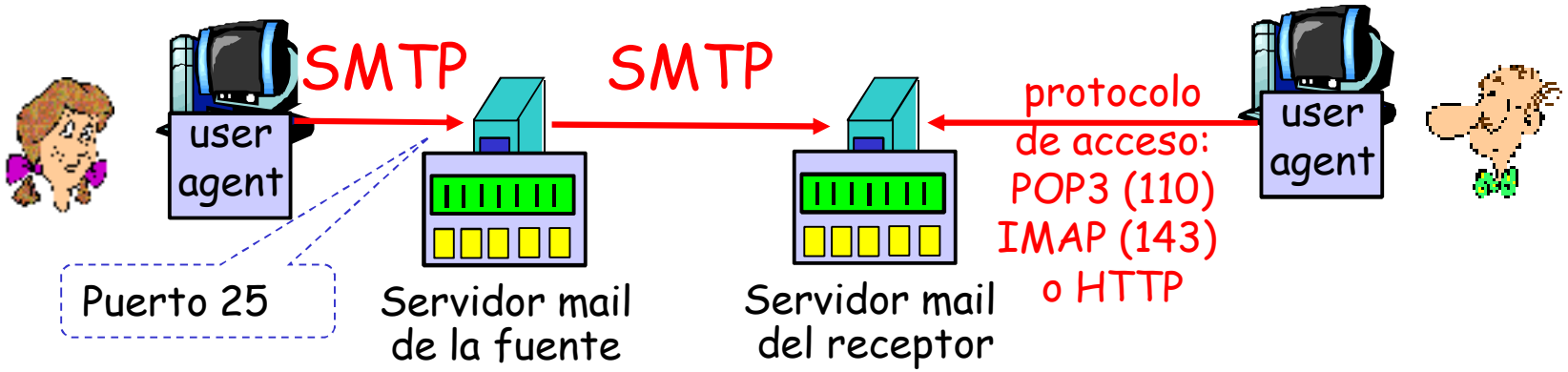
SMTP: palabras finales

- SMTP usa conexiones persistentes
- SMTP requiere que el mensaje (encabezado y cuerpo) sean en ASCII de 7-bits
- Servidor SMTP usa `CRLF.CRLF` para terminar el mensaje; es decir, una línea con sólo un punto en ella.

Comparación con HTTP:

- HTTP: pull (saca contenido desde servidor)
- SMTP: push (pone contenido en servidor)
- Ambos tienen interacción comando/respuesta en ASCII, y tienen códigos de status
- HTTP: cada objeto es encapsulado en su propio mensaje
- SMTP: múltiples objetos son enviados en un mensaje multiparte

Protocolos de acceso de correo



- SMTP: permite envío y almacenamiento de correo en servidor del destinatario
- Protocolo de acceso a correo: permite extraer correo desde el servidor
  - POP: Post Office Protocol [RFC 1939]
    - autenticación (agent <-->server) y bajada
  - IMAP: Internet Mail Access Protocol [RFC 1730]
    - Más características (más complejo)
    - Permite manipulación de los mensajes almacenados en el servidor
  - HTTP: Hotmail , Yahoo! Mail, etc.

Protocolo POP3

Fase de autorización

- Comandos del cliente:
  - `user`: declara username
  - `pass`: password
- Respuestas del servidor:
  - +OK
  - -ERR

Fase transaccional, cliente:

- `list`: lista números de mensajes
- `retr`: extrae mensajes por su número
- `dele`: borra
- `quit`

```
S: +OK POP3 server ready
C: user bob
S: +OK
C: pass hungry
S: +OK user successfully logged on
```

```
C: list
S: 1 498
S: 2 912
S: .
C: retr 1
S: <message 1 contents>
S: .
C: dele 1
C: retr 2
S: <message 1 contents>
S: .
C: dele 2
C: quit
S: +OK POP3 server signing off
```

Tamaño del mensaje

POP3 (más) e IMAP

Más sobre POP3

- Ejemplo previo usa modo "bajar y borrar".
- Bob no puede releer el correo si cambia el cliente
- "bajada y conserva": obtiene copia de los mensajes en diferentes clientes.
- POP3 no mantiene el estado de una sesión a otra ("stateless")

IMAP

- Puede mantener los mensajes en el servidor
- Permite que el usuario organice sus correos en carpetas
- IMAP mantiene el estado del usuario de una sesión a otra:
  - Nombre de carpetas mapeo entre Ids (identificadores) de mensajes y nombres de carpetas.

Origen de Web Mail

- Diciembre 1995: Dos personas aparecen con la idea frente a un inversionista. Formaron Hotmail.
- Tres empleados y 14 pasantes desarrollaron y lanzaron la primera versión 7 meses más tarde (Julio 1996).
- En menos de 1 año y medio Hotmail tenía 12 millones de cuentas y fue comprado por Microsoft en 400 millones de dólares.
- El éxito se logra por haber sido los primeros y por tratarse de una aplicación que se difunde sola.

Capítulo 2: Capa Aplicación

2.1 Principios de las aplicaciones de red

2.2 Web y HTTP

2.3 FTP

2.4 Correo Electrónico
SMTP, POP3, IMAP

2.5 DNS

DNS: Domain Name System (Sistema de nombres de dominio)

Personas: muchos identificadores:

- DNI, CUIT, CUIL, nombre, Pasaporte

Host y router en Internet:

- Dirección IP (32 bit) - usada para direccionar datagramas (ideal para router por ser máquina)
- "nombre", e.g., www.yahoo.com - son usados por humanos

Pregunta: ¿Quién mapea entre nombres y direcciones IP?

Domain Name System:

- *Base de datos distribuida* implementada en una jerarquía de muchos *servidores de nombres*
- *Protocolo de capa aplicación* permite a host, routers, y servidores de nombre comunicarse para *resolver* nombres (traducción nombre/dirección)
- No está orientado al uso directo de los usuarios, ellos usan nombres.
  - DNS es función central de la Internet implementada como protocolo de capa aplicación
  - La idea de diseño de Internet es dejar la complejidad en la "periferia" de la red.

DNS

Servicios DNS

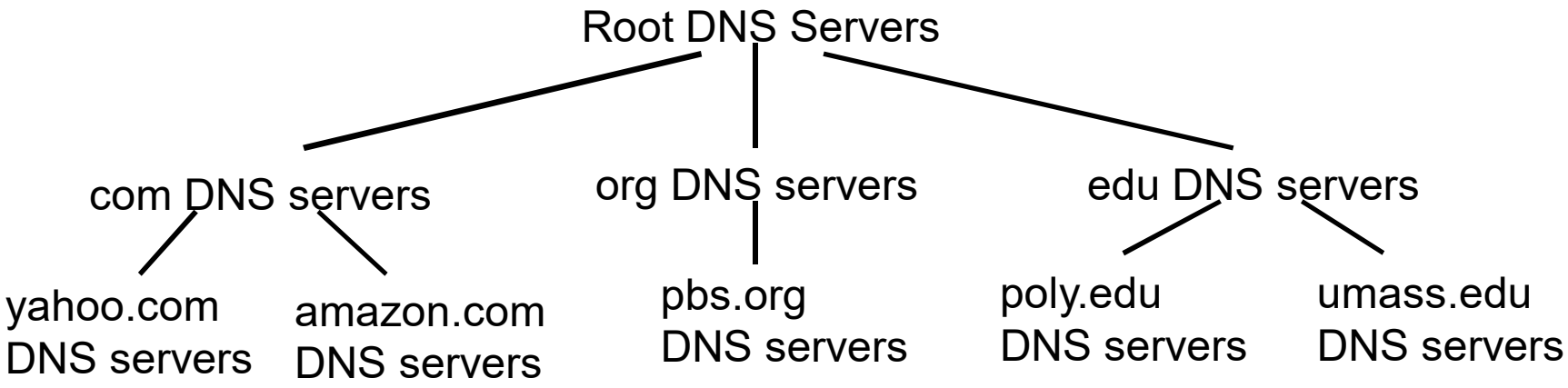
- Traducción de nombre de host a dirección IP
- Alias para host
  - Nombre canónico y alias
  - Nombre canónico: CNAME en RFC 1035
- Alias para servidor de correo
- Distribución de carga
  - Servidores Web replicados: conjunto de direcciones IP para un nombre canónico (e.g. relay1.west-coast.amazon.com), servidor DNS rota entre direcciones IP

¿Por qué no centralizar DNS?

- Sería punto único de falla.
- Volumen de tráfico, muchos necesitan el DNS
- Sería una base de datos centralizada distante con grandes retardos de acceso.
- Mantención, es mejor que cada dominio gestione sus nombres

Porque no sería *escalable!*

Base de datos jerárquica y distribuida

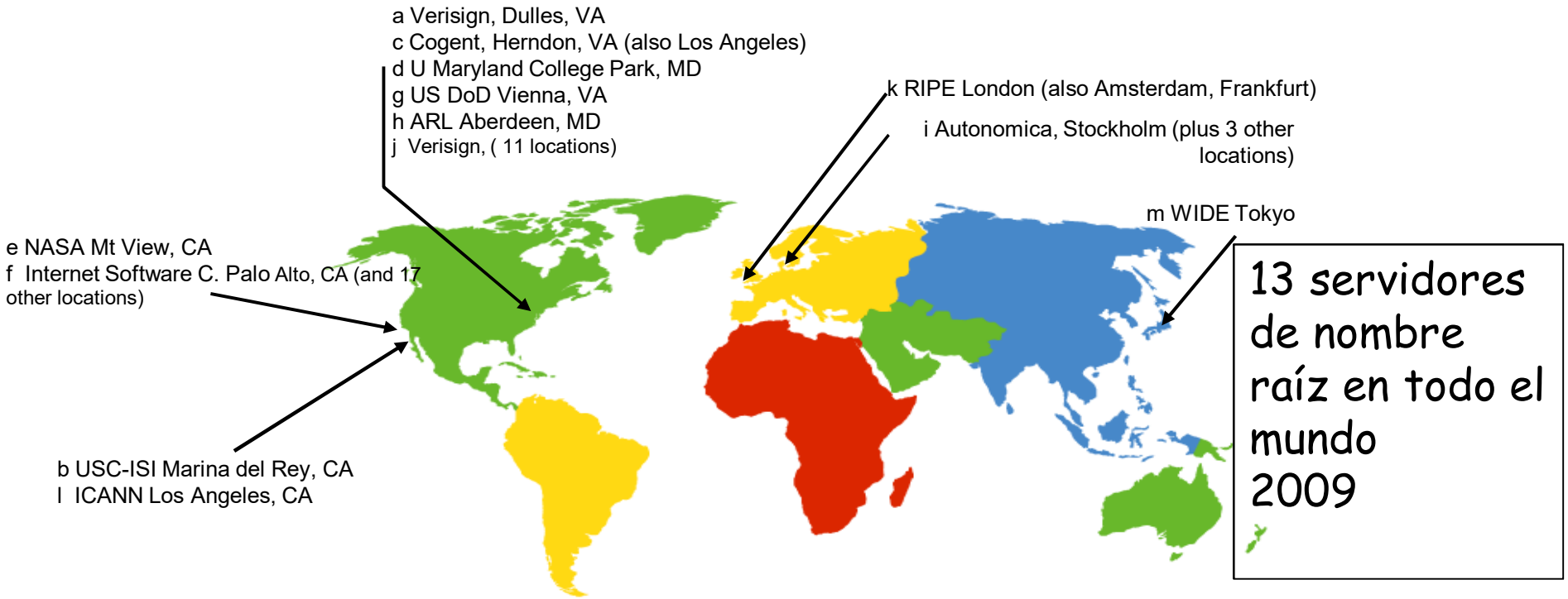


Cliente desea IP de www.amazon.com; 1ra aprox. :

- Cliente consulta al servidor raíz para encontrar servidor DNS de com
- Cliente consulta servidor DNS TLD (Top Level Domain) de com para obtener servidor DNS de amazon.com
- Cliente consulta servidor DNS amazon.com para obtener dirección IP de www.amazon.com

DNS: servidores de nombre en raíz

- Sus direcciones IPs están contenidas en el software DNS.
- Son 13 distribuidos en todo el mundo.
- Su ubicación se puede ver en: http://www.root-servers.org/
- Son contactados cuando un servidor DNS local no conoce la IP de quien maneja un dominio de nivel superior (TLD)



TLD y Servidores Autoritarios

- **Top-level domain (TLD) servers:** responsable por com, org, net, edu, etc., y todos los dominios superiores de cada país: uk, fr, ca, jp, cl, etc..
  - Network solutions mantiene servidores para el TLD de com
  - Red de Interconexión Universitaria (http://www.riu.edu.ar/) para el TLD de edu.ar
  - Nic.Ar (network information center) para el TLD de ar (www.nic.ar)
- **Servidores DNS autoritarios:** son servidores DNS de las organizaciones y proveen mapeos autoritarios entre hostname e IP (e.g., Web y mail).
  - Éstos pueden ser mantenidos por la organización o el proveedor de servicio

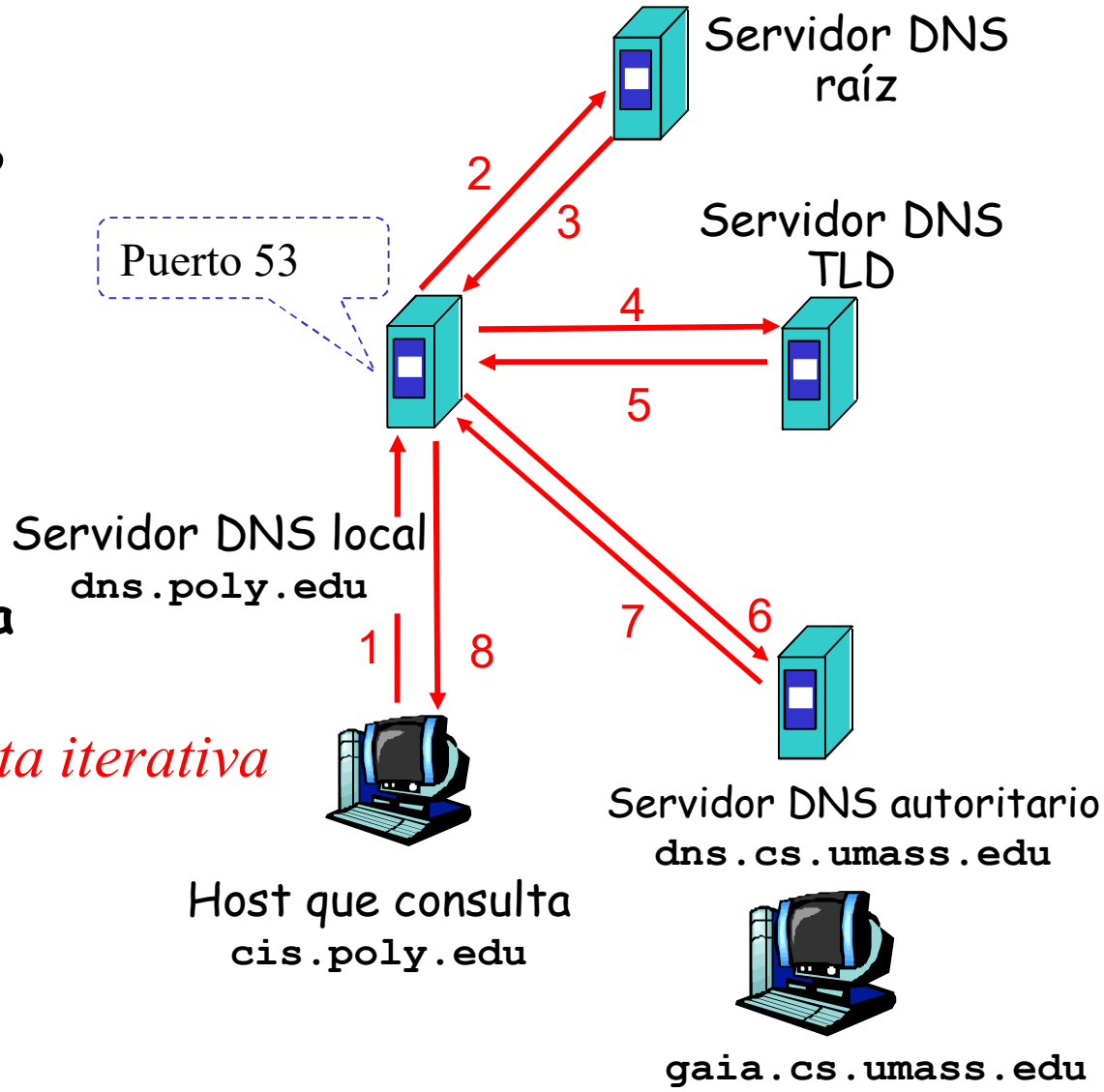
Servidor de nombre local

- No pertenece estrictamente a la jerarquía
- Cada ISP (ISP residencial, compañía, universidad) tiene uno.
  - También son llamados "servidor de nombre por omisión" (default name server)
- Cuando un host hace una consulta DNS, ésta es enviada a su servidor DNS local
  - Actúa como proxy, re-envía consulta dentro de la jerarquía.

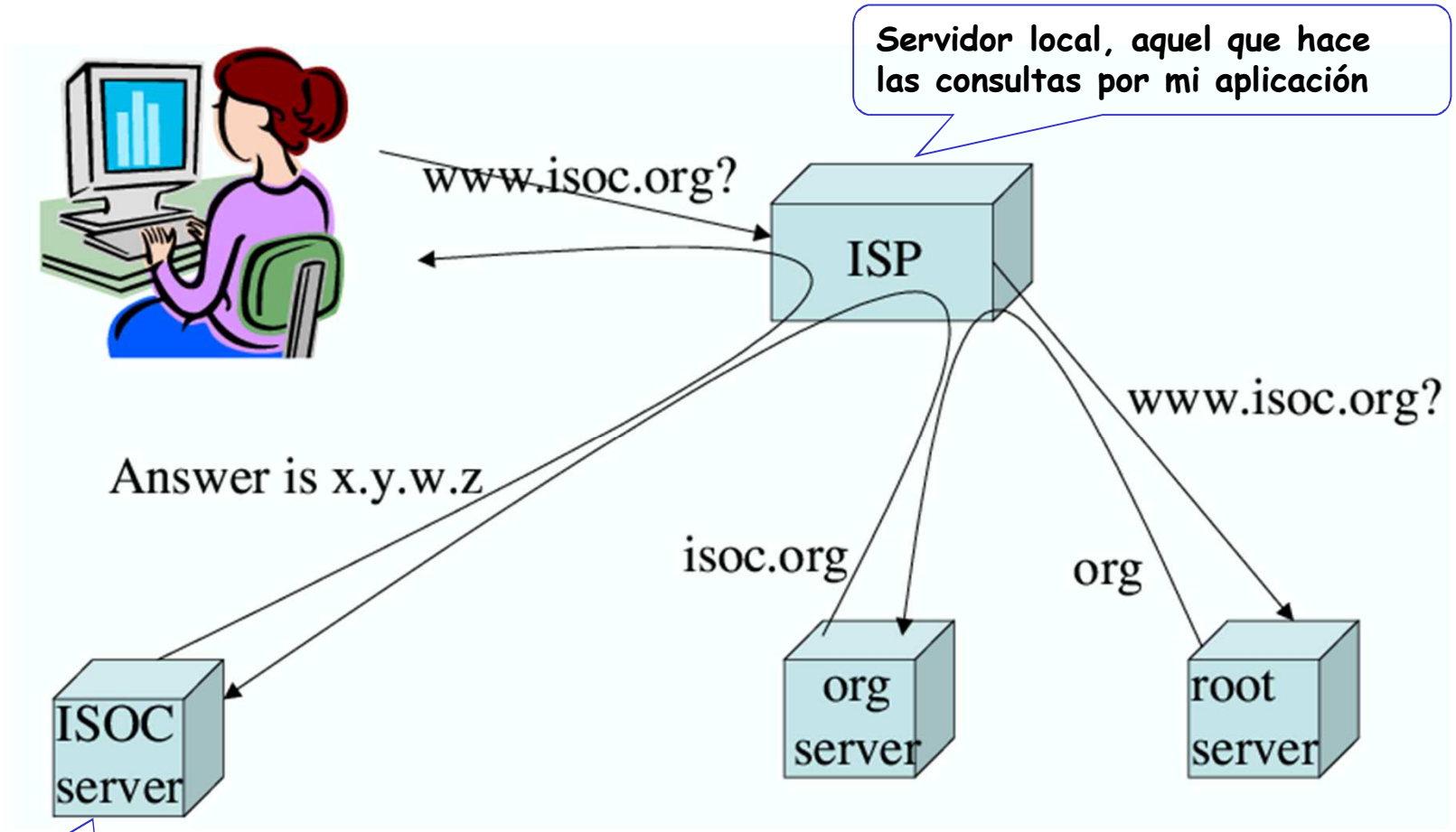
Ejemplo 1

Consulta iterativa:

- Host en cis.poly.edu quiere la dirección IP de gaia.cs.umass.edu
- Servidor contactado responde con el nombre del servidor a contactar
- "Yo no conozco este nombre, pero pregunta a este servidor"



Ejemplo 2



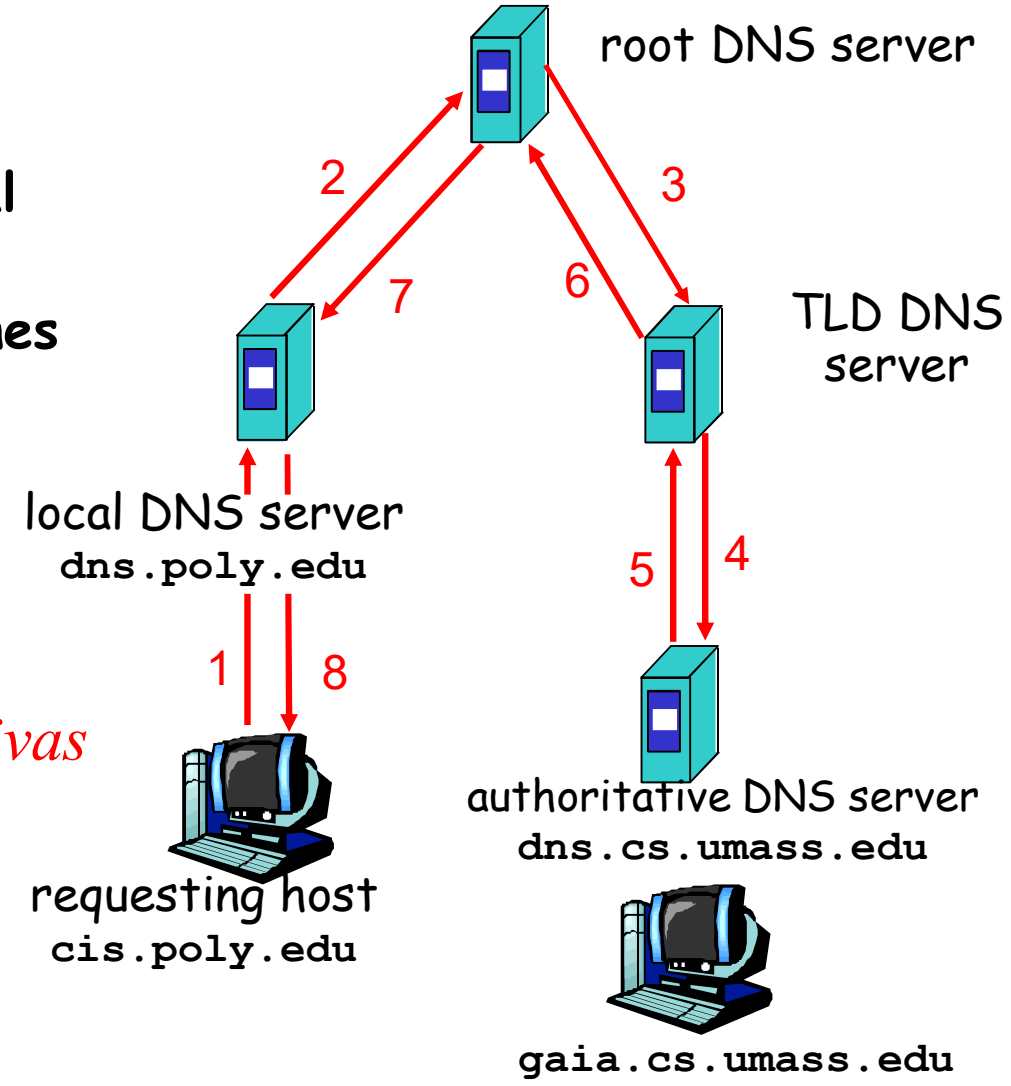
Consulta iterativa

Servidor autoritario, aquel que define el mapeo nombre <-> IP

Consultas Recursivas

Consulta recursiva :

- Pone la carga de la resolución de nombre al servidor contactado.
- ¿Qué pasa en situaciones de alta carga?



Consultas Recursivas

Un Tip !

Es común que las máquinas tengan asignados alias; por ejemplo profesores.elo.utfsm.cl es un alias para deneb.elo.utfsm.cl. ¿Expliqué por qué no conviene usar profesores.elo.utfsm.cl como nombre canónico de la máquina?

- Usando profesores.elo.utfsm.cl como alias es posible configurar otra máquina para reemplazar el servidor WEB y una vez que esté lista, sólo hacemos el cambios del alias en el DNS para que los accesos futuros se dirijan al nuevo servidor. Esta operación resulta transparente para los usuarios. Si fuera nombre canónico, mientras configuramos el nuevo servidor WEB, no podríamos tener dos máquinas con igual nombre.

DNS: Cache y actualización de registros

- Una vez que un servidor de nombre conoce un mapeo, éste *guarda* (caches) el mapeo
  - Las entradas del cache expiran (desaparecen) después de algún tiempo
  - Servidores TLD típicamente están en cache de los servidores de nombre locales
    - Así los servidores de nombre raíz no son visitados con frecuencia

Registros DNS

DNS: es una base de datos distribuida que almacena registros de recursos (resource records, RR)

Formato RR: `(name, value, type, ttl)`

- **Type=A**
  - `name` es un hostname (nombre real o canónico)
  - `value` es una dirección IP
- **Type=NS**
  - `name` es un dominio (e.g. foo.com)
  - `value` es la dirección IP (nombre) del servidor autoritario que sabe cómo obtener las direcciones IP de este dominio.

Registros DNS

DNS: es una base de datos distribuida que almacena registros de recursos (resource records, RR)

Formato RR: `(name, value, type, ttl)`

- **Type=CNAME**
  - name **es un alias para algún nombre real (indicado en type A)**
  - **www.ibm.com es realmente servereast.backup2.ibm.com**
  - value **es el nombre real (canónico)**
- **Type=MX**
  - value **es el nombre del servidor de correo asociado con** name

Inserción de registros en DNS

- Ejemplo: Recién se crea una empresa "Network Utopia"
- Debemos registrar el nombre networkuptopia.com.ar en un administrador de dominio (e.g., Network Solutions)
  - Necesitamos proveer el nombre y la dirección IP de nuestro servidor de nombre autoritario (primario y segundario)
  - Administrador del dominio inserta dos RRs en el servidor TLD com:

```
(networkutopia.com.ar, dns1.networkutopia.com.ar, NS)
(dns1.networkutopia.com.ar, 212.212.212.1, A)
```

- Incorporar en el servidor autoritario un registro Tipo A para www.networkuptopia.com.ar y un registro Tipo MX para networkutopia.com.ar
- En Argentina debemos acceder a Nic.Ar para arrendar un nombre de dominio.

Pregunta:

Explique por qué los resultados de varios PING a www.youtube.com muestran direcciones IPs distintas:

```
marra@noc03:~$ ping www.youtube.com
PING youtube-ui.l.google.com (74.125.224.45) 56(84) bytes of
data.
64 bytes from 74.125.224.45: icmp_seq=1 ttl=52 time=162 ms
.... Luego:
marra@noc03:~$ ping www.youtube.com
PING youtube-ui.l.google.com (74.125.224.42) 56(84) bytes of
data.
64 bytes from 74.125.224.42: icmp_seq=1 ttl=52 time=160 ms
.... Luego:
marra@noc03:~$ ping www.youtube.com
PING youtube-ui.l.google.com (74.125.224.44) 56(84) bytes of
data.
64 bytes from 74.125.224.44: icmp_seq=1 ttl=52 time=175 ms
...
```

Explique cómo esto se hace posible. ?

Pregunta:

- El ping a un mismo nombre lógico condujo a tres máquinas distintas por ello tres IPs distintas. Esto se explica porque el servicio de youtube es atendido por un conjunto de máquinas para poner servir a más usuarios a la vez.
- Esto es posible gracias al servidor DNS. Cuando el ping consulta por la IP de www.youtube.com, el servidor que maneja este nombre identifica la máquina adecuada de entre el conjunto para atender la petición y retorna esa dirección IP.